

Preprocessing — Data Cleaning, Train/Test Datasets, Data Annotation, and Overfitting vs Underfitting

Table of Contents

1. Overview: What is Preprocessing?
2. Training Data — Definition, Types, and How It Works
3. Types of Training Data: Structured, Unstructured, Semi-Structured
4. Testing Data — Definition and Purpose
5. How Training and Testing Work Together
6. Challenges in Creating Training Data
7. Data Annotation — Types and Examples
8. Why Separate Training and Testing Data?
9. Overfitting and Underfitting
10. Quick Reference Comparison Tables

Chapter 1: Overview — What is Preprocessing?

1.1 Simple Definition

Preprocessing is the process of preparing raw data before giving it to a machine learning model.

Raw data is messy. It can contain errors, missing values, or irrelevant information. If you feed bad data to a model, the model will give bad results.

Preprocessing fixes this problem.

1.2 Topics in Preprocessing (This Chapter)

- Data Cleaning — Removing errors and fixing bad data (basic cleaning already done).
- Train/Test Data — Splitting data into two parts for training and evaluation.

Simple Analogy:

Think of preprocessing like preparing ingredients before cooking.

You wash vegetables (data cleaning) and separate them into groups (train/test split) before cooking (training the model).

A good cook never skips preparation. A good data scientist never skips preprocessing.

1.3 Why Preprocessing Matters

- Bad data produces bad model predictions — this is called Garbage In, Garbage Out (GIGO).
 - Preprocessing improves accuracy, reliability, and fairness of the ML model.
 - It reduces the chance of overfitting and underfitting (explained later).
-

Topic	One-Line Explanation
Data Collection & Sourcing	Gathering raw data from different sources for model training.
Data Curation & Profiling	Organizing, indexing, and analyzing data to detect issues and improve accessibility.
Data Cleansing & Quality Management	Fixing missing values, errors, duplicates, and inconsistencies in the dataset.
Handling Outliers & Noise	Detecting and managing abnormal or random data points to improve model accuracy.
Data Annotation & Labeling	Assigning correct labels or tags to raw data for supervised learning tasks.
Establishing Annotation Guidelines	Creating clear rules and standards for consistent and accurate data labeling.
Categorical Encoding	Transforming categorical data into numerical form for machine learning algorithms.
Feature Scaling	Normalizing or standardizing numerical values into a similar range.
Feature Selection	Choosing the most important features and removing irrelevant ones.
Dimensionality Reduction	Reducing the number of features while preserving important information.
Handling Imbalanced Data	Balancing unequal class distributions to prevent biased predictions.
Data Augmentation	Creating modified copies of data to increase dataset size and diversity.
Train/Test Data Splitting	Dividing data into training and testing sets for model evaluation.
Cross-Validation Setup	Using multiple validation rounds to reduce overfitting and improve reliability.

Chapter 2: Training Data

2.1 Simple Definition

Training data is the dataset that is used to teach (train) a machine learning model.

It is the most important input in the learning process. The model studies this data, finds patterns, and builds a decision-making system based on those patterns.

Key Point:

Training data is LARGER than testing data.

More data means the model learns more patterns and makes more accurate predictions.

Typically, 75-80% of your total data is used for training.

2.2 Structure of Training Data

Training data is made up of input-output pairs:

Component	Explanation
Input (also called Features or Predictors)	The information given to the model. Example: age, blood pressure, weight.
Output (also called Label or Target)	The answer the model should predict. Example: 1 (heart attack possible) or 0 (no heart attack).
Example (one data point)	One row in a dataset. It contains all the features and the label.

2.3 Types of Information in Training Data

- Images — used in image classification tasks.
- Text — used in natural language processing (NLP) tasks.
- Numerical values — used in regression and classification.
- Sensor readings — used in IoT and real-time monitoring systems.

Real-World Example:

In image classification for self-driving cars, training data contains thousands of road images.

Each image is paired with a label: 'pedestrian', 'car', 'traffic light', etc.

The model learns to recognize these objects from the labeled examples.

2.4 How Training Data Teaches the ML Model

The learning process happens in the following way:

1. The algorithm receives training data.
2. It analyzes patterns inside the data.
3. It adjusts its internal parameters (settings) to reduce the gap between its predictions and the correct labels.
4. This process repeats many times. Each time, the model gets better.
5. The model gradually builds a representation of the underlying patterns in the data.
6. Finally, the model can apply what it learned to new, unseen data.

2.5 Why Training Data Quality Matters

The quality of training data directly affects the quality of predictions. Consider the following:

- If training data contains errors, the model learns incorrect patterns.
- If training data is biased (not representative), the model will be biased too.
- If training data is too small, the model will not learn enough patterns.
- A large, clean, and representative training dataset leads to accurate and reliable models.

Important Note:

Without an accurate and representative training dataset, machine learning models may develop biases and oversights.

These biases restrict the model's ability to generalize to unfamiliar examples.

Always verify the quality and diversity of your training data.

2.6 Key Points Summary — Training Data

Aspect	Details
Purpose	Teach the ML model to recognize patterns

Aspect	Details
Size	Usually 75-80% of total dataset
Content	Input features + output labels (in supervised learning)
Quality requirement	Clean, accurate, large, and representative
Effect of poor quality	Biased, inaccurate, or unreliable model predictions

Chapter 3: Types of Training Data

3.1 Overview

Training data comes in three main types. Each type has a different structure, different sources, and different use cases.

Type	Structure	Examples
Structured	Fixed format: rows and columns	Databases, spreadsheets, CSV files
Unstructured	No fixed format	Images, videos, text, audio
Semi-Structured	Partial structure, some flexibility	JSON, XML, log files, social media posts

3.2 Structured Data

Definition

Structured data is organized in a fixed format — rows and columns. It follows a defined schema (a blueprint that describes how data is arranged). Machines can easily read and analyze it.

Examples

- Excel spreadsheets with rows for customers and columns for name, age, and city.
- A hospital database with patient records.
- A financial database with stock market prices.

Sources of Structured Data

- Databases (SQL databases like MySQL, PostgreSQL)
- Customer Relationship Management (CRM) systems
- Financial records and banking systems
- APIs and Web Services (data delivered in JSON or XML format)
- Internet of Things (IoT) Devices (smart home sensors, industrial machines)

Applications of Structured Data

Field	Use Case
Finance	Analyzing stock market trends and forecasting market changes

Field	Use Case
Healthcare	Electronic Health Records (EHRs) for patient diagnosis and decision support
SEO / Web	Microdata tags in HTML help search engines understand webpage content better
Retail	Predictive analytics for customer purchasing behavior
Banking	Fraud detection by analyzing transaction patterns

Simple Example:

A bank wants to detect fraud. It stores transaction data in a structured database.

Each row is one transaction. Columns include: amount, location, time, account number.

The ML model reads these rows and learns what a fraudulent transaction looks like.

3.3 Unstructured Data

Definition

Unstructured data is information that has no predefined format or organization. It does not fit neatly into rows and columns.

It is often qualitative (descriptive) in nature rather than numerical.

Types of Unstructured Data

- Text — customer reviews, emails, articles, social media posts
- Images — photos, medical scans, satellite images
- Videos — surveillance footage, YouTube videos
- Audio recordings — voice messages, call recordings
- Sensor readings — when stored without a schema
- Multimedia files — PDFs, presentations

Why is Unstructured Data Challenging?

Machines cannot directly read and understand a photo or an audio file.

Special techniques like Computer Vision (for images) and Natural Language Processing (for text) are needed.

These techniques convert unstructured data into a format the model can process.

Real-World Example

A social media company wants to detect hate speech. The data consists of millions of text posts — unstructured data.

NLP techniques convert the text into numerical vectors. The model then classifies each post as harmful or safe.

3.4 Semi-Structured Data

Definition

Semi-structured data sits between structured and unstructured data. It has some organization but still allows flexibility in format.

It has a partial structure — for example, data may be grouped by tags or keys, but the exact fields can vary.

Sources of Semi-Structured Data

- Web applications with user interactions (login data, clicks)
- API responses (JSON or XML format)
- Log files from servers or applications
- Internet of Things (IoT) Devices
- Social media platforms (posts with metadata)
- Legal documents and survey responses

Example of Semi-Structured Data (JSON)

JSON Example:

```
{ "name": "Ahmed", "age": 25, "city": "Dhaka", "skills": ["Python", "ML"] }
```

This data has keys (name, age) but the 'skills' field can hold a variable number of values.

This flexibility is what makes it semi-structured.

3.5 Comparison: All Three Data Types

Feature	Structured	Unstructured	Semi-Structured
Format	Fixed (rows/columns)	No fixed format	Partial format
Storage	Relational databases	File systems, cloud storage	NoSQL databases, files
Examples	CSV, SQL tables, Excel	Images, video, audio, text	JSON, XML, log files
Processing Difficulty	Easy	Hard (needs special tools)	Moderate
ML Use Case	Classification, regression	Computer Vision, NLP	Web analytics, IoT
Data Size	Smaller but organized	Very large	Medium

Important Keywords:

Structured Data | Unstructured Data | Semi-Structured Data | Schema | Features
| Labels | Training Data | JSON | XML

Practice Questions

1. What is the difference between structured and unstructured data? Give one example of each.
 2. A hospital stores patient records in an Excel file. What type of data is this? Why?
 3. Why is unstructured data harder to process than structured data?
 4. Name two sources of semi-structured data.
 5. A social media company wants to build a hate speech detector. What type of training data will they use?
-

Chapter 4: Testing Data (Test Dataset)

4.1 Simple Definition

Test data is a separate portion of the original dataset. It is used to evaluate how well the trained model performs on new, unseen data.

The model has NEVER seen the test data during training. This is very important. If the model sees the test data during training, the evaluation will be dishonest.

4.2 Key Properties of Test Data

- It is independent of the training dataset.
- It has similar types of features and class distribution as the training data.
- It represents real-world scenarios the model will face after deployment.
- It is used as a benchmark to measure model performance.
- It should be large enough to give meaningful and reliable evaluation results.
- It should represent or be part of the original dataset — not a random subset with different properties.

4.3 Size of Test Data

Standard Rule:

The test dataset is approximately 20-25% of the total original data.

If you have 1,000 data records, approximately 200-250 are used for testing.

The remaining 750-800 records are used for training.

4.4 Training vs Testing Data — Comparison Table

Training Data	Testing Data
Used to TRAIN (teach) the model	Used to EVALUATE the model's performance
Model sees this data and learns from it	Model has NEVER seen this data before
Larger portion — about 75-80% of total data	Smaller portion — about 20-25% of total data
Contains labels (in supervised learning)	Contains labels only for comparison after prediction

Training Data	Testing Data
Adjusts the model's internal parameters	Does NOT change the model's parameters
More data = better learning	Must be representative and meaningful

4.5 Real-World Example: Heart Attack Prediction

[Insert Image from Slide 21 Here]

Image Explanation: Slide 21 shows a table with 13 medical attributes such as age, sex, blood pressure (trtbps), cholesterol (chol), and other health indicators. The last column is the output label — 0 (no heart attack) or 1 (heart attack possible). This is a classic supervised learning dataset used in medical ML applications.

Consider a heart attack prediction dataset:

- Each row represents one patient.
- The 13 columns (features) include: age, sex, blood pressure, cholesterol, and other health measurements.
- The last column (label/output) is binary: 0 = no heart attack risk, 1 = heart attack possible.

How it works:

Training: The model is trained on 75-80% of patients. It learns which combination of features leads to a heart attack.

Testing: The model is tested on the remaining 20-25% of patients.

The test data has the labels (answers), but the model makes predictions WITHOUT seeing them.

The predictions are then compared to the actual labels to measure accuracy.

[Insert Image from Slide 22 Here]

Image Explanation: Slide 22 likely shows the same heart attack dataset or a visual split of the data into training and testing portions. This reinforces the concept of the 75-80% / 20-25% split rule.

Chapter 5: How Training and Testing Work Together

5.1 The Complete ML Workflow

Machine learning algorithms learn from past observations (training data) and then make predictions on new data (test data). The complete process works in three steps:

[Insert Image from Slide 14 Here]

Image Explanation: Slide 14 contains a diagram showing the three-step workflow: Feed (training data), Define (tagging and transforming data), and Test (evaluating with unseen data). The diagram also shows a model being ready after testing. This is a critical visual to understand the full ML training-to-testing pipeline.

Step 1 — FEED: Provide Training Data

The first step is to feed the model with training input data. This includes all the features (input columns) and their corresponding labels (output column).

The model receives this data and begins learning.

Step 2 — DEFINE: Tag and Transform

In supervised learning, the training data is tagged with correct output labels.

The model transforms the training data into numerical vectors or data features that it can process mathematically.

This step is the actual learning process — the model adjusts its internal parameters based on the patterns it finds.

Step 3 — TEST: Evaluate with Unseen Data

After training is complete, the model is tested using the test dataset (unseen data).

This step checks whether the model has truly learned the patterns — or just memorized the training data.

A model that performs well on test data is said to generalize well.

5.2 What Does 'Generalization' Mean?

Generalization Definition:

Generalization is the ability of a model to perform well on NEW, UNSEEN data.

A model that only performs well on training data but fails on test data has NOT generalized.

Good generalization = the model truly learned the patterns, not just memorized the examples.

5.3 Why Must Training and Testing Data Be Separate?

- If the model is tested on the same data it was trained on, it will appear very accurate — but this is dishonest.
 - The model may have simply memorized the training examples without truly understanding the patterns.
 - Separate test data reveals the true performance of the model.
 - It simulates a real-world scenario where the model encounters new, previously unseen data.
-

Chapter 6: Challenges in Creating Training Data

6.1 Overview

Creating high-quality training data is one of the most difficult parts of an ML project. There are three main categories of challenges:

- Data Collection
- Data Quality Management
- Data Annotation

6.2 Challenge 1: Data Collection

Difficulties in Sourcing and Collecting Data

Finding the right data for a specific ML task is not always easy. Data may be:

- Scattered across multiple sources
- Locked in proprietary databases
- Too expensive to collect at scale
- Subject to privacy and legal restrictions

Managing Data Quality

Raw data almost always contains errors and inconsistencies:

- Missing values (some rows have empty fields)
- Duplicate records (same data entered twice)
- Incorrect values (typos, wrong measurements)
- Inconsistent formats (date as '01/05/2024' vs '2024-05-01')

Solution: Data profiling (finding problems) + Data cleansing (fixing problems).

Finding Relevant Data

Not all data is useful for a specific ML task. Data scientists must curate the data — selecting only what is relevant.

Solution: Create a data catalog and searchable indexes to simplify data discovery.

Dealing with Big Data

Modern ML projects often involve massive volumes of data — both unstructured and semi-structured. This adds complexity in both collection and processing.

Solution: Use a data lake — a centralized repository where large volumes of raw data are stored until needed.

6.3 Challenge 2: Data Annotation

What is Data Annotation?

Data annotation is the process of labeling data with relevant tags to make it understandable for computers. It tells the model what each piece of data represents.

Simple Analogy:

Imagine showing a child thousands of fruit photos without labels.

The child cannot learn which is an apple and which is a banana.

Data annotation is like writing 'Apple' and 'Banana' on each photo so the model can learn.

Why Data Annotation is Important

Purpose	Explanation
Training ML Models	ML models need labeled data to learn and make accurate predictions in supervised learning.
Enhancing Data Understanding	Labels add context to raw data. For example, marking objects in images helps algorithms recognize them accurately.
Validating Model Performance	Annotated data is also used as ground truth to evaluate how well the model performs.

Chapter 7: Types of Data Annotation

7.1 Overview of Annotation Types

Data annotation covers four main types. Each type is used for different kinds of data and different ML tasks.

7.2 Image Annotation

Image annotation involves labeling objects, regions, or features within an image.

Annotation Technique	Description
Bounding Boxes	A rectangle is drawn around each object (e.g., a car or a person)
Polygons	An irregular shape drawn around the exact boundary of an object
Key Point Annotations	Marks specific points (e.g., facial landmarks: eyes, nose, mouth)
Semantic Segmentation	Every pixel in the image is labeled with a class (e.g., road, sky, car)

Use Case: Self-driving cars use image annotation to detect pedestrians, vehicles, and traffic signs.

7.3 Text Annotation

Text annotation focuses on labeling or tagging textual data.

- Named Entity Recognition (NER) — Identifying names, places, dates in text.
- Sentiment Analysis — Labeling text as positive, negative, or neutral.
- Part-of-Speech Tagging — Labeling each word as noun, verb, adjective, etc.
- Text Classification — Assigning a category to a document (e.g., 'spam' or 'not spam').

7.4 Audio Annotation

Audio annotation involves labeling and transcribing audio data.

- Speech Recognition — Converting spoken words to text with labels.
- Speaker Identification — Labeling which person is speaking in a recording.
- Emotion Detection — Labeling the emotional tone of speech (happy, angry, sad).

7.5 Video Annotation

Video annotation involves labeling objects, actions, or events across multiple frames in a video.

- Activity Recognition — Labeling what action is happening (e.g., 'running', 'sitting').
- Object Tracking — Following an object (e.g., a car) across multiple video frames.
- Event Detection — Identifying when a specific event occurs in the video.

7.6 Annotation Types Summary Table

Type	Data Format	Common Tasks
Image Annotation	Photos, satellite images, medical scans	Object detection, face recognition, autonomous vehicles
Text Annotation	Reviews, articles, social media	Sentiment analysis, chatbots, spam detection
Audio Annotation	Voice recordings, calls	Voice assistants (Siri, Alexa), call analysis
Video Annotation	Surveillance footage, sports videos	Action recognition, security systems, sports analytics

7.7 Detailed Example: Image Annotation for Autonomous Vehicles

[Insert Image from Slide 18 Here]

Image Explanation: Slide 18 describes the data annotation process for self-driving cars. It includes a photo of a city road with pedestrians and vehicles. This image represents the kind of raw training data collected for autonomous vehicle systems — streets, intersections, pedestrians, and cars captured from a car's camera.

Autonomous vehicles are an excellent example of how image annotation works in practice. Let us walk through the full process:

Step 1 — Dataset Collection

A large dataset of road images is collected. The images include pedestrians, vehicles, traffic signs, road markings, and other objects encountered on real roads.

Step 2 — Annotation Guidelines

Before annotation begins, a set of guidelines is created. These guidelines define:

- Which objects should be labeled (e.g., 'car', 'pedestrian', 'bicycle', 'stop sign', 'traffic light').
- How each object should be annotated (e.g., bounding box for cars, polygon for road boundaries).
- Edge cases — how to handle partially visible objects or unusual scenarios.

Step 3 — Annotators' Role

Skilled and trained annotators review each image and apply the correct labels. Their work is critical because the model learns directly from their annotations.

Annotation Challenges for Autonomous Vehicles

[Insert Image from Slide 19 Here]

Image Explanation: Slide 19 describes annotation challenges specific to self-driving car scenarios. It emphasizes real-world complexity: objects at unusual angles, partial occlusion (objects hidden behind other objects), and rare edge cases that annotators must handle carefully.

Challenge	Explanation
Object Variations	Objects can appear in different sizes, shapes, orientations, and levels of visibility. A partially hidden pedestrian is still a pedestrian and must be labeled correctly.
Ambiguity	Some objects are difficult to classify definitively. Annotators must use expert judgment to assign the best possible label.
Edge Cases	Self-driving cars encounter rare and unusual scenarios. Annotations must cover a wide range of situations to ensure robustness — for example, a bicycle on the wrong side of the road.

Important Keywords:

Data Annotation | Image Annotation | Text Annotation | Bounding Box | Semantic Segmentation | Named Entity Recognition | Sentiment Analysis | Edge Cases | Ground Truth

Practice Questions

6. What is data annotation? Why is it important for supervised machine learning?
 7. What is the difference between image annotation and text annotation?
 8. Explain what 'semantic segmentation' means in image annotation.
 9. List two challenges faced when annotating data for autonomous vehicles.
 10. A company builds a voice assistant (like Siri). What type of data annotation do they need?
-

Chapter 8: Why Separate Training and Testing Data?

[Insert Image from Slide 20 Here]

Image Explanation: Slide 20 asks the question: 'Why is it important to use separate training and testing data?' This is a conceptual slide meant to make the student think critically before the answer is revealed. The answer follows in the subsequent slides.

8.1 The Core Reason

The goal of machine learning is NOT to memorize training data. The goal is to LEARN patterns that apply to new, unseen data.

If we test the model on the same data it was trained on, we cannot know if it truly learned patterns or just memorized answers.

Real-World Analogy:

Imagine a student studies 100 exam questions at home.

If you test them with the same 100 questions, they will score 100% — but they may have just memorized the answers.

If you give them 20 NEW but similar questions, you test their real understanding.

That is exactly what test data does for ML models.

8.2 Key Reasons for Separation

- **Honest Evaluation** — Test data gives an unbiased, honest measure of model performance.
 - **Generalization Check** — It verifies that the model works on real-world, new data.
 - **Detecting Overfitting** — If training accuracy is high but test accuracy is low, the model is overfitted.
 - **Preventing Data Leakage** — Mixing training and test data causes data leakage, which leads to overly optimistic (fake) performance scores.
-

Chapter 9: Overfitting and Underfitting

9.1 Overview

Overfitting and underfitting are the two most common and important problems in machine learning. Understanding them is essential for building good models.

[Insert Image from Slide 26 Here]

Image Explanation: Slide 26 contains a diagram showing three scenarios side by side: Underfitting (too simple), Balanced (just right), and Overfitting (too complex). The dots (data points) and the curve (decision boundary) illustrate how a model can fail in both directions. This is one of the most important visual concepts in ML.

9.2 Overfitting

Simple Definition

Overfitting happens when the model performs very well on training data but fails on test (new) data.

The model has memorized the training data, including its noise and random fluctuations. It has not learned the true underlying pattern.

Key Symptom of Overfitting:

Training Accuracy = Very HIGH (e.g., 97%)

Testing Accuracy = Very LOW (e.g., 68%)

The large gap between training and testing accuracy is the sign of overfitting.

Why Does Overfitting Happen?

- The model is too complex — it has too many parameters relative to the amount of training data.
- The training data is too small — the model overfits to a few examples.
- The model trains for too long — it starts memorizing noise in the data.

What is Noise in Data?

Noise refers to random errors or irrelevant information in the training data. A good model should ignore noise and learn only the true patterns. An overfitted model treats noise as if it were a real pattern.

How to Prevent Overfitting

- Cross-Validation — Split training data into multiple folds and validate the model repeatedly.
- Early Stopping — Stop the training process before the model starts memorizing noise.
- Regularization — Add a penalty to the model for excessive complexity (L1 and L2 regularization).
- More Training Data — More data reduces the chance of memorization.
- Dropout (in neural networks) — Randomly disable some neurons during training to prevent reliance on specific patterns.

9.3 Underfitting

Simple Definition

Underfitting happens when the model is too simple. It fails to learn even the basic patterns in the training data. As a result, it performs poorly on BOTH training data AND test data.

Key Symptom of Underfitting:

Training Accuracy = Very LOW (e.g., 60%)

Testing Accuracy = Very LOW (e.g., 55%)

Both accuracies are low. The model has not learned enough from the training data.

Why Does Underfitting Happen?

- The model is too simple — for example, using a straight line to fit data that follows a curve.
- Training time is too short — the model did not have enough time to learn.
- Too few features — the model lacks the information it needs to make good decisions.
- Too much regularization — over-penalizing the model prevents it from learning properly.

How to Fix Underfitting

- Use a more complex model with more parameters.
- Train for more epochs (more iterations).
- Add more relevant features to the dataset.
- Reduce regularization if it is too strong.

9.4 Overfitting vs Underfitting vs Balanced — Full Comparison

Aspect	Overfitting	Underfitting	Balanced (Good Fit)
Also called	High variance	High bias	Low bias, low variance
Training Accuracy	Very High (e.g., 97%)	Low (e.g., 60%)	Reasonably High
Testing Accuracy	Very Low (e.g., 68%)	Low (e.g., 55%)	Close to training accuracy
Model Complexity	Too Complex	Too Simple	Just Right
Cause	Memorizing noise and specific examples	Failing to learn patterns	Learning true patterns
Generalization	Poor — fails on new data	Poor — fails everywhere	Good — works on new data
Solution	Regularization, more data, early stopping	More complex model, more features, more training	Fine-tune hyperparameters

9.5 Numerical Examples

Example 1 — Overfitting

Training Accuracy: 97% | Testing Accuracy: 68%

The gap between training (97%) and testing (68%) accuracy is very large.

This is a clear sign of OVERFITTING.

The model memorized the training data but cannot generalize to new examples.

Example 2 — Underfitting

Training Accuracy: 60% | Testing Accuracy: 55%

Both accuracies are very low.

The model could not even learn the training data properly.

This is a clear sign of UNDERFITTING.

What Does a Good (Balanced) Model Look Like?

Good Model Example:

Training Accuracy: 91% | Testing Accuracy: 88%

The training accuracy is high, and the testing accuracy is close to it.

The small gap means the model generalized well — it learned real patterns, not noise.

Important Keywords:

Overfitting | Underfitting | Generalization | Noise | Training Accuracy | Testing Accuracy | Cross-Validation | Regularization | Early Stopping | Balanced Model

Practice Questions

11. What is overfitting? How do you identify it from training and testing accuracy?
 12. A model has 98% training accuracy and 61% testing accuracy. What problem does this indicate?
 13. A model has 59% training accuracy and 57% testing accuracy. What problem does this indicate?
 14. List three methods to prevent overfitting.
 15. Explain the difference between overfitting and underfitting using a real-world analogy.
-

Chapter 10: Quick Reference — All Comparison Tables

10.1 Training Data vs Testing Data

Training Data	Testing Data
Fits (trains) the model	Evaluates the model
75-80% of total data	20-25% of total data
Model sees labels during learning	Labels revealed only for comparison after prediction
More data = better model	Must be representative and sufficiently large
Adjusts model parameters	Does NOT change model parameters

10.2 Structured vs Unstructured vs Semi-Structured Data

Feature	Structured	Unstructured	Semi-Structured
Format	Fixed (rows/columns)	No format	Partial format
Examples	SQL table, Excel	Image, video, text	JSON, XML, log files
Storage	Relational DB	File storage/cloud	NoSQL DB
Processing	Easy	Needs special tools	Moderate effort
ML Tasks	Classification, regression	Computer Vision, NLP	Web analytics, IoT

10.3 Overfitting vs Underfitting

Overfitting	Underfitting
Very high training accuracy	Low training accuracy
Very low testing accuracy	Low testing accuracy
Model is too complex	Model is too simple
Memorizes noise and training examples	Fails to learn basic patterns
Fix: Regularization, early stopping, more data	Fix: More complex model, more training, more features

10.4 Data Annotation Types

Annotation Type	Used For
Image Annotation	Bounding boxes, segmentation — used in self-driving cars, medical imaging
Text Annotation	NER, sentiment analysis — used in chatbots, spam filters
Audio Annotation	Speech recognition, emotion detection — used in voice assistants
Video Annotation	Action recognition, object tracking — used in surveillance, sports

10.5 Challenges in Training Data

Challenge Category	Key Issues
Data Collection	Hard to source, quality issues, finding relevant data, managing Big Data
Data Quality	Errors, duplicates, missing values, inconsistent formats
Data Annotation	Expensive, time-consuming, requires expertise, ambiguity in labeling
Big Data	Volume and variety of data makes processing complex

Final Summary — Key Takeaways

1. Preprocessing prepares raw data for ML. It includes data cleaning and splitting into train/test sets.

2. Training data teaches the model. It is larger (75-80%) and contains input-output pairs.

3. Testing data evaluates the model honestly on unseen data (20-25% of total data).

4. Data types: Structured (fixed format), Unstructured (no format), Semi-Structured (partial format).

5. Data Annotation is labeling data for supervised learning. Types: Image, Text, Audio, Video annotation.

6. Overfitting = model too complex, performs well on training data but poorly on test data.

7. Underfitting = model too simple, performs poorly on both training and test data.

8. A balanced model generalizes well — performs similarly on both training and test data.

9. Quality training data is critical. Garbage In = Garbage Out.

10. Always separate training and testing data to get an honest evaluation of model performance.

End of Chapter Notes — Machine Learning Preprocessing and Data Management